CONNECT money

BO Team: Jira to Google Sheets Automation

Technical and User Documentation

How the sheets are built, how to read them, and how to keep them running

Created by	Mohamed Magdy
Company	Connect Money
Team	Back Office (BO), Jira project `L1`
Version	1.1
Date	5 October 2026

Contents

1 Introduction **3**
1.1 What this system does . . . 3
1.2 The two sheets . . . 3
1.3 Who should read which part . . . 3
1.4 Conventions . . . 3

2 System at a Glance **4**
2.1 The moving parts . . . 4
2.2 Components . . . 4
2.3 What happens in one refresh . . . 5

3 Where the Data Comes From **5**
3.1 Jira . . . 5
3.2 Jira statuses used . . . 6
3.3 The three source sheets . . . 6

4 The BO Tickets (Auto) Sheet **6**
4.1 Which tickets appear . . . 6
4.2 Tabs . . . 7
4.3 Columns of the ticket tabs . . . 7
4.4 Extra columns copied from the source sheets . . . 7
4.5 The dropdowns . . . 8
4.6 How a ticket is routed to a tab . . . 8
4.7 The Backlog tab . . . 9
4.8 The Productivity tab (BO sheet) . . . 9

5 The In Progress Sheet **9**
5.1 Tabs . . . 9
5.2 Which tickets appear and who is left out . . . 10
5.3 Columns of the team tabs . . . 10
5.4 Which team has the problem . . . 10
5.5 The Status dropdown and what “Done” means . . . 11
5.6 Counters in the header row . . . 11
5.7 The Pending Customer Response tab . . . 11
5.8 The Productivity tab (In Progress sheet) . . . 12
5.9 The hidden Done Log tab . . . 12
5.10 Colour guide . . . 13

6 How the Sheets Stay Up to Date **13**
6.1 When a refresh happens . . . 13
6.2 What a refresh changes . . . 13
6.3 Running time and free minutes . . . 14
6.4 Why the timer lives in Google and not in GitHub . . . 14
6.5 Starting a refresh by hand . . . 14

7 Setup and Configuration (for the Maintainer) 14
7.1 Repository contents . . . 14
7.2 Settings . . . 15
7.3 Building it from scratch . . . 16

8 Everyday Maintenance 16

9 Troubleshooting 17

10 Security and Data Handling 18

11 Limits, Decisions, and Open Items 19
11.1 Decisions made along the way . . . 19
11.2 Known limits . . . 19
11.3 Open items . . . 19

12 A Ticket's Journey: Putting It Together 20

13 Project History 20

A Glossary 21

B Quick Checklists 21

C Google Apps Script (timer) 22

D GitHub Workflow 23

1 Introduction

1.1 What this system does

The BO team receives its work as Jira tickets in the project **L1 Support (BO)** (key `L1`). Before this project, the team tracked those tickets by hand in Google Sheets. This system copies the tickets from Jira into two Google Sheets *automatically*, keeps them up to date during the day and night, and adds the numbers the team lead needs: backlog per contact type, tickets per team and per person, time spent in each status, and productivity.

In one sentence: **Jira stays the source of truth; the two Google Sheets are a live, easy-to-read, easy-to-edit view of it.**

1.2 The two sheets

Sheet	Purpose
BO Tickets (Auto)	New work. Shows the tickets that are still waiting to be picked up (Jira status *To Do* or *Clients To Do*), split by team (MDB, Installment, Product, Others), with a backlog table and a productivity summary.
In Progress	Work in flight. Shows the tickets that the team is working on, tickets waiting for the customer, and each person's workload and results.

1.3 Who should read which part

Reader	What you need	Chapters
BO team member	How to read the sheets and use the dropdowns	4, 5
Team lead / manager	What each number means and how reliable it is	4, 5, 9, 10
Maintainer	How it runs, how to change it, how to fix it	2, 6, 7, 8, 9
Anyone new	Terms used in this document	Appendix A

1.4 Conventions

Names of tabs, columns, and dropdown values are written in **bold**. File names, settings, and commands are written in `typewriter font`. All times are in **Cairo time** (Africa/Cairo) unless stated otherwise.

2 System at a Glance

2.1 The moving parts

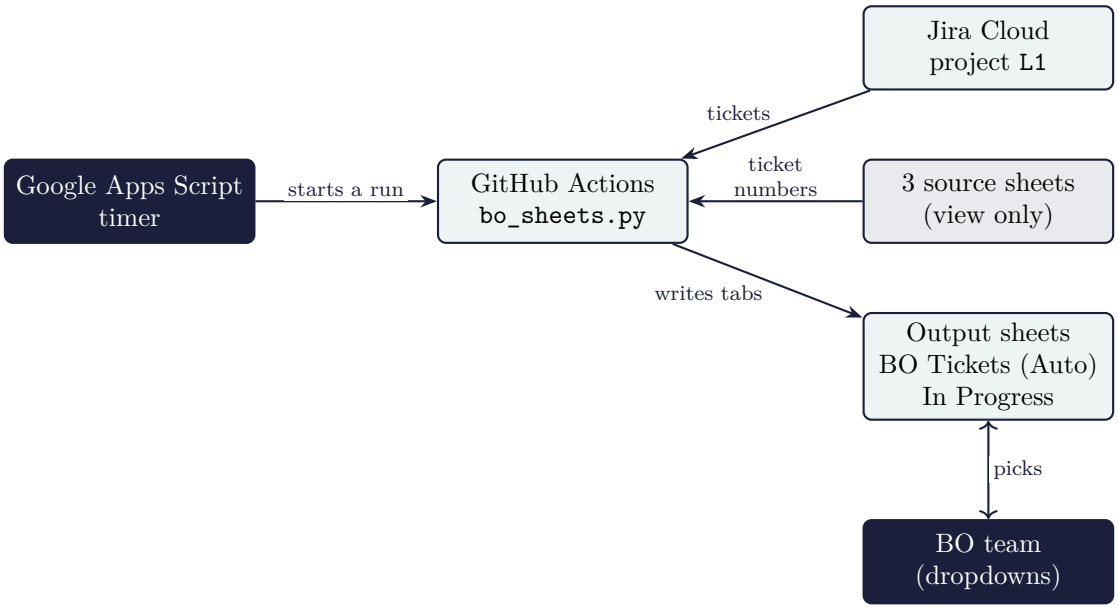


Figure 1: How the parts connect. Arrows show the direction of data.

2.2 Components

Component	Role
Jira Cloud (`connect-money.atlassian.net`)	Holds the tickets. Read only: the system never changes Jira.
`bo_sheets.py`	The program. Reads Jira and the source sheets, builds the tabs, and writes them to Google Sheets.
GitHub repository (private)	Stores the code and runs it on GitHub's servers through the workflow `.github/workflows/sync.yml`.
Google Apps Script (`trigger.gs`)	A timer that asks GitHub to start a run at the right times. It does no data work itself.
Google service account	A robot Google account that the program uses to read and write the sheets.
Output sheets	**BO Tickets (Auto)** and **In Progress**. Editable by the robot and the team.
Source sheets	Three existing team sheets that say which team a ticket belongs to. Read only.

2.3 What happens in one refresh

1. The timer in Google Apps Script calls GitHub and starts the workflow.
2. GitHub starts a small Linux machine, downloads the code, installs the Python libraries, and runs `bo_sheets.py`.
3. The program asks Jira for the relevant tickets, and at the same time reads the three source sheets.
4. It decides which tab each ticket belongs to, and computes every column.
5. It rewrites the tabs in both output sheets, keeping the dropdown choices people made, and rebuilds colours.
6. The machine shuts down. A normal run takes about 30 seconds on GitHub (measured on 5 October 2026: 32 seconds).

Nothing is stored outside Google and GitHub

The program keeps no database. Everything it needs to remember lives in the sheets themselves, including a hidden tab called **Done Log** (see Section 5.9).

3 Where the Data Comes From

3.1 Jira

The program reads the Jira project `L1` (*L1 Support (BO)*). It uses these ticket properties:

Jira property	How it is used
Key (for example `L1-15393`)	Identifies the ticket; becomes a link in the sheet.
Status	Decides which sheet and tab the ticket appears in.
Assignee	Shown as the **Person** in the In Progress sheet.
Created	Start of the “days since arrived” counter.
Request Type (L1)	The “contact type”. Used for routing and for the Backlog table.
Summary	Contains the customer ticket reference at the end, for example `| CCS-15449`.
Status history (changelog)	Tells when a ticket entered its current status, and who closed it.
Related Tickets Status	Lists the linked tickets in other teams’ projects and their status (used for *With Team*).

3.2 Jira statuses used

Meaning in the sheets	Jira statuses
New / untouched	`To Do`, `Clients To Do`
In progress with BO	`In Progress`, `Open`, `Reopened`, `Work in progress`
Waiting for the customer	`Pending Customer Response`
Closed	Every status Jira classifies as "Done" (for example `Done`, `Resolved BO`, `Resolved by OPS`, `Resolved by Collection`)
Everything else	Pending with other teams (for example `Pending Tech (Lucky)`, `Pending BM`); treated as *Pending* in the Backlog table and not shown on the ticket tabs

Jira's Resolution field is not used

In this Jira, tickets are closed by moving them to a Done-type status; the *Resolution* field is empty. The system therefore recognises closed tickets by their status, not by resolution.

3.3 The three source sheets

The queues (MDB, Installment, Product) are *not* fields in Jira. The team keeps them in three Google Sheets. The system reads those sheets (view only) to learn which team a ticket belongs to.

Team tab	Source sheet	Tabs read
MDB	*MDP 2025/2026* (the sheet called "MDB" by the team)	All tabs
Installment	*Credit Sheet*	Only *Callcenter Cases 2026 V2*
Product	*FO/BO 2026 - 2027*	All tabs (it has one)

A ticket "belongs" to a source sheet when its ticket number appears in any cell of that sheet. The program recognises the number formats `L1-1234`, `CCS-1234`, `CC-1234`, and `CC2-1234`, and compares both the Jira key and the customer reference from the summary. Cells longer than 40 characters are ignored, so a number mentioned inside a long comment does not count.

4 The BO Tickets (Auto) Sheet

4.1 Which tickets appear

Only tickets that satisfy *all* of these:

- They are in the Jira project `L1`.
- Their status is `To Do` or `Clients To Do`.
- They were created on or after **1 October 2026**.

When a ticket is picked up and its Jira status changes (for example to *In Progress*), it disappears from this sheet at the next refresh and appears in the **In Progress** sheet instead.

4.2 Tabs

Tab	Content
Backlog	Closed / Pending / Untouched counts for each contact type (Section 4.7).
Productivity	Live counts per team and per person (Section 4.8).
MDB	To-do tickets that belong to the MDB team.
Installment	To-do tickets that belong to the Installment team.
Product	To-do tickets that belong to the Product team.
Others	To-do tickets that match no team. Check it regularly.

4.3 Columns of the ticket tabs

Column	Meaning
Key	The Jira ticket number.
Link	Opens the ticket in Jira.
Request Type	The contact type, as set in Jira.
Customer Ticket	The customer support reference taken from the end of the summary (for example `CCS-15449`).
Status	The Jira status (*To Do* or *Clients To Do*).
Assignee	**Dropdown** with the team names. Chosen by hand in the sheet. Not connected to Jira.
Created	When the ticket was created in Jira.
Updated	When the ticket was last changed in Jira (any comment, field change, or status change).
Resolved	**Dropdown**: *Solved*, *Pending for customer response*, *On hold*. Chosen by hand.

The Summary column is deliberately not shown on this sheet.

4.4 Extra columns copied from the source sheets

To the right of *Resolved*, each tab gets columns that the program copies from the team's source sheet. The row is found by the ticket number, as in Section 4.6.

Tab	Columns added (and where they come from)
MDB	*Card #*, *Acc No*, *Description*, *Amount*, *Disputed Amount If Partial*, *Project*, *BO Owner*, *Validation*: the same-named columns of the MDP sheet.
Installment	*Comment*: the *First Comment* column of the Credit Sheet.
Product	*Comment*: the *Comment* column of the FO/BO sheet (often an escalation ticket such as `TEC-14867`).
Others	*Comment*: no source sheet, so it starts empty and can be typed by hand.

- A new ticket usually is not in the source sheet yet, because people fill those sheets by hand.

Its cells stay empty and fill in automatically at the first refresh after the row appears.

- A *Comment* typed by hand in this sheet is kept as long as the source sheet has no comment for that ticket. When the source sheet does have one, the source wins and replaces what was typed.
- *Card #* and *Acc No* are stored as text, so numbers such as `0538` keep their leading zero.

4.5 The dropdowns

Assignee Mego, Helana, Abdallah, Heba, Reem, Samy, Bassem, Mayar. The names are used exactly as written; the system never tries to match them to Jira users. The column starts empty.

Resolved Solved Pending for customer response On hold.

Each choice is coloured so the sheet can be scanned quickly. Choices are remembered by ticket key: when the sheet refreshes, a choice stays with its ticket even if the ticket moves to a different tab.

Dropdown choices are only in the sheet

Choosing *Solved* here does **not** change the ticket in Jira. It is a working note for the team. Once the ticket leaves the To Do list in Jira, it leaves this sheet and its choices go with it.

4.6 How a ticket is routed to a tab

For every ticket the program follows the same three steps, in this order, and uses the first one that matches:

1. **Keywords in the Request Type.** If the Request Type contains one of the words below, the ticket goes to that tab. The comparison ignores capital letters.
2. **The source sheets.** If the ticket number is found in the MDB, then Installment, then Product source sheet, it goes to that tab.
3. **Others.** Anything else.

Tab	Keywords (a Request Type containing any of these)
Installment	`installment`, `unable to pay due`
MDB	`refund transaction`, `purchase failed`, `deposit didn't reflect`, `pos transfer`, `unable to deposit`, `unable to withdraw`
Product	`different balance`

Why keywords come first

New tickets are often not yet written in the source sheets, because those sheets are filled in by hand. Keywords let the system route a new ticket immediately. The source sheets are the fallback for types that no keyword covers.

4.7 The Backlog tab

The table is modelled on the backlog table in the team's reporting. It covers *every* ticket created since 1 October 2026 (not only the To Do ones), grouped by Request Type (the contact type), and sorted by the largest *Closed* count first.

Column	Counts the tickets whose...
Closed	status is a Done-type status.
Pending	status is anything else that is not To Do or Clients To Do.
Untouched	status is `To Do` or `Clients To Do` (nobody has started).
Total	is the sum of the three.

Each of the first three columns is shaded from white to a stronger colour: the darker the cell, the larger the number. Use **Untouched** and **Pending** to see where the backlog is building up.

4.8 The Productivity tab (BO sheet)

This tab is made of **live formulas** that read the other tabs, so the numbers change the moment someone changes a dropdown, without waiting for a refresh.

Left table: per team. One row each for MDB, Installment, Product and Others, and a Total row.

- **Total Tickets**: how many tickets are in that team's tab.
- **Solved**, **Pending for customer response**, **On hold**: how many tickets have that *Resolved* choice.
- **Not Set Yet**: tickets with no *Resolved* choice (total minus the three above).

Right table: per person. One row for each name in the Assignee dropdown, showing how many tickets that person has in MDB, Installment, Product and Others, and a Total. The last row, **No one picked yet**, counts tickets that have no Assignee choice.

5 The In Progress Sheet

5.1 Tabs

Tab	Content
MDB, **Installment**, **Product**, **Others**	Tickets currently in progress with the BO team, routed by the same rules as the BO sheet (Section 4.6).
Pending Customer Response	Tickets sent back to the customer, with how long they have been waiting.
Productivity	One row per person with their counts (Section 5.8).
Done Log (hidden)	The memory of who closed which ticket (Section 5.9).

5.2 Which tickets appear and who is left out

- Tickets whose Jira status is `In Progress`, `Open`, `Reopened`, or `Work in progress` appear on the team tabs, with no limit on age. A ticket stuck for a month still shows.
- Tickets in `Pending Customer Response` appear on their own tab.
- Tickets assigned to the following people are **left out of the whole sheet**: *loay.khaled*, *nourhan.khaled*, *mohanad.maher*, and *Abdullah Mubark*.
- Unassigned tickets are shown with the person *Unassigned*.

5.3 Columns of the team tabs

Column	Meaning
Key, Link, Request Type, Summary	Ticket number, link to Jira, contact type, and the ticket summary.
Person	The Jira assignee.
Status	**Dropdown**: *In Progress* or *Done*. The team sets it by hand. New tickets start as In Progress.
Jira Status	The real status in Jira.
In Status Since	When the ticket entered its current Jira status.
Hours	Live counter: hours since "In Status Since".
Created	When the ticket was created.
Days Since Arrived	Live counter: days since "Created".
Days In Progress	Live counter: days since "In Status Since".
With Team	Which team has the problem now (Section 5.4).
Related Tickets (Jira)	The linked tickets in other teams' Jira projects and their status, as Jira lists them.

How the live counters work

Hours, *Days Since Arrived* and *Days In Progress* are spreadsheet formulas based on the current time. They keep increasing between refreshes without any help from the program. A ticket created at 09:00 shows "0.5" days at 21:00.

5.4 Which team has the problem

Jira keeps a field called *Related Tickets Status*. When a BO ticket is escalated, the field lists the linked ticket in the other team's project and that ticket's status, for example `Operations (OPS-1934): Pending CS`. The program reads this field and fills two columns on every ticket tab of the In Progress sheet:

- **Related Tickets (Jira)**: exactly what Jira lists, shortened to *team, ticket, status*.
- **With Team**: who has the problem now, worked out with this rule:

What Jira shows	With Team
No related ticket	*BO* (or *Customer* on the Pending Customer Response tab)
A related ticket whose status does *not* mention CS, for example *In Progress (OPS)*	the name of that team, for example *Operations*
Every related ticket has a CS status, for example *Pending CS* or *PENDING RESPONSE (CS)*	*BO (reply from Operations)*: the other team has answered and is waiting for us

This rule is an interpretation

With Team is based on the status wording "CS" meaning that the other team is waiting for Customer Support. If the other teams use their statuses differently, the rule can be changed in one place in the program. The column *Related Tickets (Jira)* always shows the raw facts so the result can be checked.

Tickets that sit in an escalation status of their own (for example *Pending Tech (Lucky)* or *Pending Collection*) are not shown on this sheet; those statuses already name the team.

5.5 The Status dropdown and what "Done" means

- The program does **not** add finished tickets to the team tabs automatically. A ticket finished in Jira simply leaves the tab.
- Instead, the team can mark a ticket *Done* by hand while it is still open in Jira. The choice is kept through refreshes, and the ticket is counted as Done in the Productivity tab and in the tab's counters.
- Setting *Done* does not change Jira. When the ticket is closed in Jira, it leaves the tab.

5.6 Counters in the header row

The top row of each tab also shows live totals, to the right of the table, so they are always visible:

- Team tabs: **Total**, **In Progress**, and **Done** (based on the Status dropdown).
- Pending Customer Response: **Total** and **Overdue**.
- Productivity: **All people** (the grand total of cases).

The filter arrows cover only the ticket columns, so sorting or filtering a table never moves the counters.

5.7 The Pending Customer Response tab

Lists the tickets in the Jira status *Pending Customer Response*. It has the same columns as the team tabs, with these differences:

- **Sent To Customer** replaces "In Status Since": the time the ticket was put into that status.
- **Days Waiting For Customer** replaces "Days In Progress".

- **Over 48h?** shows **OVERDUE** when the ticket has waited more than **48 hours**. This is the *only* place in the sheet with an overdue flag. The team tabs have none.

5.8 The Productivity tab (In Progress sheet)

One row per person, listed once, largest total first. Colours work like the Backlog table.

Column	Meaning
Person	The person's Jira name.
In Progress	Their open tickets whose Status dropdown is not *Done*.
Done (last 30 days)	Tickets they moved to a Done-type status in Jira in the last 30 days, plus open tickets they have marked *Done* in the sheet.
Pending Customer	Their tickets that are waiting for the customer.
Total Cases	In Progress + Done + Pending Customer.

How to read the Done numbers

"Done" is credited to the person who *moved the ticket* to the Done status in Jira, which may differ from the assignee. If someone closes a large batch of old tickets at once, their number jumps. Read large numbers together with the dates, and do not use them as a measure of individual speed without checking.

The excluded people (Section 5.2) do not appear. Tickets with no assignee are not listed in Productivity, because they belong to nobody.

5.9 The hidden Done Log tab

Reading the full Jira history of about 3 200 recently closed tickets takes over 30 seconds, which is too slow to do every run. Instead, the program keeps a hidden tab named **Done Log** with one row per closed ticket: its key, the time it was closed (as a Unix time in seconds), and who closed it. A cell beside it records the time of the last run. Each refresh only asks Jira about tickets that changed since then, adds them, removes tickets that were reopened, and drops entries older than 30 days.

Do not delete or edit the Done Log

If it is deleted, nothing breaks: the next run rebuilds it from the last 30 days, which takes about 40 seconds once. Editing it by hand will change the Done counts.

5.10 Colour guide

Colour	Where and what it means
Green	*Done*, *Solved*, and the Closed column.
Blue	*In Progress*.
Yellow	*Pending Customer Response*, *Pending for customer response*, and Pending columns.
Red	*On hold*, *Reopened*, the Untouched column, and **OVERDUE**.
Grey	*To Do*.
Purple	*Clients To Do*.

6 How the Sheets Stay Up to Date

6.1 When a refresh happens

A Google Apps Script timer wakes up every 30 minutes and decides whether to start a run:

Time (Cairo)	Refresh
09:00 – 22:59 (day)	every 30 minutes (about 28 runs)
23:00 – 08:59 (night)	every 2 hours: 23:00, 01:00, 03:00, 05:00, 07:00 (5 runs)

Google's timer handles the clock change at the end of October (Egypt moves from UTC+3 to UTC+2) automatically, so the schedule needs no edit. A refresh can also be started by hand (Section 6.5).

6.2 What a refresh changes

- Every tab is cleared and rewritten from fresh data.
- **Kept:** the team's dropdown choices (Assignee, Resolved, Status), matched by ticket key.
- **Rebuilt:** colours, dropdown lists, and the order of tabs.
- **Live:** all counters and totals are formulas, so they update by themselves.

If someone picks a dropdown value at the very second a refresh finishes, the choice can occasionally be lost. Check that it stuck.

6.3 Running time and free minutes

GitHub's free plan for private repositories includes **2 000 minutes per month**, and each run is rounded up to whole minutes.

Runs per day (28 day + 5 night)	33
Runs per month (30 days)	about 990
Billed minutes if each run is under 1 minute	about 990
Billed minutes if each run takes 1–2 minutes	about 1 980
Free allowance	2 000

A normal run was measured at 32 seconds, so the expected use is about 1 000 minutes a month, with a safety margin. Check *Settings* → *Billing* → *Actions* on GitHub from time to time.

6.4 Why the timer lives in Google and not in GitHub

GitHub can start workflows on a schedule of its own, and this was the first design. In practice GitHub's scheduler is *best effort*: on the first day it started only one scheduled run over more than six hours, so new tickets did not reach the sheet. The Google timer does not have this problem. The GitHub schedule was therefore removed, and the workflow now runs only when called by the Google timer or by hand.

6.5 Starting a refresh by hand

Open the repository on GitHub, choose the **Actions** tab, select **Sync Jira to Google Sheets**, and click **Run workflow**. The run appears in the list and finishes in about 30 seconds.

7 Setup and Configuration (for the Maintainer)

7.1 Repository contents

File	Purpose
`bo_sheets.py`	The main program (both sheets).
`jira_to_sheets.py`	The first, simpler exporter (one flat table). No longer used by the workflow.
`requirements.txt`	Python libraries: `requests`, `gspread`, `google-auth`.
`.github/workflows/sync.yml`	The workflow that runs the program.
`apps-script/trigger.gs`	The Google timer code.
`.env.example`	List of all settings, without secrets.
`docs/`	This document and the logo files.

7.2 Settings

Settings are environment variables. In the workflow, secrets come from GitHub's secret store and the rest are written in the workflow file.

Setting	Default	Meaning
JIRA_BASE_URL	(set in workflow)	Jira site address.
JIRA_EMAIL	(set in workflow)	Login email used with the Jira token.
JIRA_API_TOKEN	**secret**	Jira API token.
GOOGLE_SERVICE_ACCOUNT_JSON	**secret**	Full contents of the service account key.
BO_SHEET_ID	(set in workflow)	Spreadsheet ID of BO Tickets (Auto).
INPROGRESS_SHEET_ID	(set in workflow)	Spreadsheet ID of In Progress.
JIRA_PROJECT	L1	Jira project key.
BO_FROM	2026-10-01	BO sheet shows tickets created on or after this date.
BO_TODO_STATUSES	To Do, Clients To Do	Statuses that count as untouched.
INPROGRESS_STATUSES	In Progress, Open, Reopened, Work in progress	Statuses shown as in progress.
CUSTOMER_STATUS	Pending Customer Response	Status for the customer-waiting tab.
PENDING_SLA_HOURS	48	Hours before a customer-waiting ticket is OVERDUE.
PRODUCTIVITY_DAYS	30	Window for the Done counts.
EXCLUDED_PEOPLE	loay.khaled, nourhan.khaled, mohanad.maher, Abdullah Mubark	Left out of the In Progress sheet.
SHEET_TIMEZONE	Africa/Cairo	Time zone set on both sheets.
MDB_SOURCE_ID, INSTALLMENT_SOURCE_ID, INSTALLMENT_SOURCE_TAB, PRODUCT_SOURCE_ID	the three source sheets	Where the team sheets are.

7.3 Building it from scratch

1. **Jira token.** At `https://id.atlassian.com/manage-profile/security/api-tokens` create an API token for the Jira account. Choose an expiry date.
2. **Google project.** In Google Cloud Console, enable the *Google Sheets API* (and the *Google Drive API*). Create a *service account* and download its JSON key.
3. **Create the two output sheets** in a normal Google account (a service account has no storage of its own and cannot create files). Share both with the service account's email address as **Editor**.
4. **Share the three source sheets** with the same address as **Viewer**.
5. **GitHub.** Keep the repository **private**. Under *Settings* → *Secrets and variables* → *Actions* add the secrets `JIRA_API_TOKEN` and `GOOGLE_SERVICE_ACCOUNT_JSON` (paste the whole key file, from the first `{` to the last `}`, brackets included).
6. **GitHub access token for the timer.** Create a fine-grained personal access token limited to this one repository with only *Actions: Read and write*. Choose an expiry date and note it.
7. **Google timer.** In the BO sheet open *Extensions* → *Apps Script*, paste the code from Appendix C, and save. In *Project Settings* → *Script properties* add `GITHUB_TOKEN` with the token from step 6. Run `triggerSync` once and accept the permission request. Under *Triggers* add a time-driven trigger for `triggerSync`, minutes timer, every 30 minutes.
8. **Check.** The Actions tab should show a new run, and the sheets should fill.

Two expiry dates to track

The Jira token and the GitHub token both expire. The GitHub token used for the timer expires on **4 November 2026**. After expiry the timer fails quietly and the sheets stop updating. Create a new token before that date, and paste it into the Apps Script property `GITHUB_TOKEN`.

8 Everyday Maintenance

I want to...	Do this
Send a new kind of ticket to a tab	Add a keyword to `KEYWORDS` in `bo_sheets.py` (Section 4.6), or add the ticket numbers to the matching source sheet.
Check which tickets are in *Others*	Open the Others tab, read the *Request Type* column, and decide which tab each type belongs to. Then add a keyword for it.
Add or remove a name in the Assignee dropdown	Edit the list `TEAM` in `bo_sheets.py`.

I want to...	Do this
Leave another person out of the In Progress sheet	Add their Jira display name to `EXCLUDED_PEOPLE`.
Change the 48-hour limit	Set `PENDING_SLA_HOURS`.
Change the BO sheet's start date	Set `BO_FROM` (format `YYYY-MM-DD`).
Count other statuses as in progress	Set `INPROGRESS_STATUSES`.
Change how often it refreshes	Edit the day and night rules in the Apps Script (Appendix C); mind the free minutes (Section 6.3).
Refresh now	Run the workflow by hand (Section 6.5).
Rebuild the Done counts	Delete the hidden *Done Log* tab; the next run rebuilds it.
Replace an expiring token	Create a new token and replace the secret (GitHub) or the property (Apps Script).

All code changes go through the repository: edit the file, commit to a working branch, and merge into `main`. The workflow always runs the code on `main`.

9 Troubleshooting

What you see	Likely cause	What to do
Sheet is hours old, new tickets missing	The timer did not start a run, or the run failed.	Look at the repository's Actions tab. If there are no recent runs, check the Apps Script *Executions* list. Start a run by hand.
Apps Script error `404`	The GitHub token cannot see the private repository.	Check that the token's owner and repository are right and that *Actions: Read and write* is set.
Apps Script error `401` / `403`	Token wrong, expired, or lacking permission.	Create a new token.
Run fails with `429`	Google's limit on requests per minute.	The program waits 70 seconds and retries up to four times. If it keeps failing, wait and run again.
A ticket is missing from the BO sheet	It is not To Do / Clients To Do, was created before 1 October 2026, or is in a different project.	Check the ticket in Jira.

What you see	Likely cause	What to do
A ticket is in *Others*	No keyword matches and it is not in a source sheet.	Add a keyword or add it to the source sheet.
A dropdown choice vanished	The ticket left the list, or a refresh finished while it was chosen.	Choose it again; check the ticket's Jira status.
Done numbers look too high	A person closed many tickets at once.	Check dates; see Section 5.8.
Counters show an error	A tab was renamed, or a column moved.	Do not rename tabs or move columns; run a refresh to rebuild.
Run fails with a Jira error	Jira token expired or the Jira site is unavailable.	Create a new token, or wait.

The first place to look

Open the **Actions** tab in the GitHub repository. Each run shows a green tick or a red cross. Click a red run to read the exact error message.

10 Security and Data Handling

- **Secrets are never in the code.** The repository holds no token or key. The Jira token and the Google key live in GitHub's encrypted secret store. The GitHub token lives in the Apps Script's script properties.
- **The repository must stay private.** It contains the sheet addresses and the company's Jira site name.
- **Least access.** The service account can edit only the two output sheets and can only view the three source sheets. The GitHub token can only start workflows in this one repository.
- **Read only on Jira.** The program never changes a ticket.
- **Sheet privacy.** Both output sheets are restricted: only people they are shared with can open them.
- **Personal data.** The sheets show ticket numbers, request types, summaries, and staff names. They do not copy customer phone numbers or card details from the source sheets: the source sheets are read only to find ticket numbers.

Rotate credentials that were shared during development

During development, a Jira token and a service account key were pasted into a chat session. A credential that has been shared should be treated as exposed. Revoke those tokens (Jira: *Security* → *API tokens*; Google Cloud: delete and re-create the service account key) and put the new values only into the GitHub secrets.

11 Limits, Decisions, and Open Items

11.1 Decisions made along the way

- **Keywords before source sheets.** Routing by Request Type keywords gives instant results for new tickets; the hand-filled source sheets are the fallback.
- **Names used as given.** The Assignee dropdown is never matched to Jira users.
- **Done is a manual dropdown.** The In Progress tabs do not list finished tickets automatically.
- **Overdue only for customer waiting.** The 24-hour overdue flag was removed from the team tabs; the customer-waiting tab uses 48 hours.
- **One timer.** Only the Google timer starts runs, so refreshes are not doubled.

11.2 Known limits

- The sheets can be up to 30 minutes behind Jira by day and about 2 hours behind overnight.
- Dropdown choices do not write back to Jira.
- Done is credited to the person who moved the ticket, so batch closures distort individual numbers.
- Some tickets have no assignee in Jira (about 80 at the time of writing); they appear as *Unassigned* on the team tabs and are left out of Productivity.
- The Others tab depends on the keyword list; new request types land there until a keyword is added.
- The team has not yet decided where some request types belong (for example *Card is temporarily blocked* and *Credit Limit Not Activated*); they currently fall under Others on the In Progress sheet.
- Google allows about 60 reads and 60 writes per minute for this service account. A normal run uses far fewer.

11.3 Open items

1. Decide the team for the request types that currently land in *Others*.
2. Decide whether the In Progress sheet should also get the Backlog table, the Assignee dropdown, or the Resolved dropdown, as on the BO sheet.
3. Replace the GitHub token before 4 November 2026.
4. Review the monthly free-minute usage in mid-month and at the end of the month.
5. Revoke and replace the credentials that were shared during development (Chapter 10).

12 A Ticket's Journey: Putting It Together

1. **A customer contacts support.** A Jira ticket is created in project `L1` with status *To Do*.
2. **Within 30 minutes** (or 2 hours overnight) the ticket appears in the **BO Tickets (Auto)** sheet, in the tab chosen by its Request Type (for example *Lucky Refund Transaction* → **MDB**). It counts as *Untouched* in the Backlog tab.
3. **Someone takes it.** In the sheet they pick their name under *Assignee*, and the per-person table in Productivity moves by one. In Jira the ticket moves to *In Progress*.
4. **At the next refresh** the ticket leaves the BO sheet and appears in the **In Progress** sheet, on the team tab, with a live *Hours* and *Days* counter. Backlog now counts it as *Pending*.
5. **The team asks the customer for information.** In Jira the status becomes *Pending Customer Response*. The ticket moves to that tab; after 48 hours it shows **OVERDUE**.
6. **The ticket is closed in Jira** (for example *Resolved BO*). It leaves the In Progress sheet, is written to the Done Log with the name of whoever closed it, and counts toward that person's *Done* for the next 30 days. Backlog counts it as *Closed*.

13 Project History

Step	What was built
1	First exporter: all Jira tickets into one private Google Sheet.
2	Connection to Jira and Google verified; service account sharing set up.
3	BO sheet and In Progress sheet created, with Pending Customer Response and Productivity.
4	Routing into MDB, Installment, Product; Others tab; source sheets read for matching.
5	Dropdowns (Assignee, Resolved) with colours; Summary and Reporter columns removed.
6	Automatic refresh with GitHub Actions; schedule tuned to working hours and the free minutes.
7	In Progress sheet reorganised into team tabs; Status dropdown; 48-hour customer flag; one-row-per-person productivity; Backlog table.
8	Day counters (days since arrived, days in progress).
9	Reliability fix: GitHub's scheduler skipped runs, so the Google timer now starts every run.
10	Speed-up: Done Log, parallel reads, batched writes (run time from about 80 seconds to about 30 seconds).
11	BO Productivity tab; live totals in the In Progress tabs; this document.
12	MDP case columns on the BO MDB tab; Comment column on the other BO tabs; *With Team* and *Related Tickets* on the In Progress sheet.

A Glossary

BO	Back Office, the team this system serves.
L1	The key of the Jira project *L1 Support (BO)*.
MDB / MDP	The team's name for the queue that is tracked in the sheet *MDP 2025/2026*.
Request Type	The Jira field that says what the customer contacted support about (also called the contact type).
Customer ticket (CCS)	The reference of the original customer support ticket; written at the end of a Jira summary.
Backlog	Work that has arrived but is not finished.
Untouched	A ticket nobody has started (status To Do or Clients To Do).
Pending	A ticket that is open but not untouched.
Overdue	A customer-waiting ticket that has waited more than 48 hours.
Workflow	A set of steps that GitHub runs on a server; here, the refresh.
Secret	A password or key stored encrypted so that nobody can read it back.
Service account	A robot Google account used by programs.
Apps Script	Google's built-in scripting tool for Google Sheets.
JQL	Jira's query language, used to ask Jira for a list of tickets.
Dropdown	A cell that lets you pick from a fixed list.

B Quick Checklists

Every day (team lead)

- ☐ The *Updated* column in the BO sheet shows recent times.
- ☐ The *Others* tab is empty or understood.
- ☐ Pending Customer Response: any **OVERDUE** rows to chase.

Every week (maintainer)

- ☐ Latest runs in the Actions tab are green and appear on schedule.
- ☐ Request types in *Others* have been assigned keywords.

Every month

- ☐ GitHub billing page: Actions minutes used are well under 2 000.
- ☐ Token expiry dates (Jira, GitHub) are more than two weeks away.

C Google Apps Script (timer)

This is the full code of the timer. It contains no secret: the token is read from the script property `GITHUB_TOKEN`.

```
const REPO = 'mazenDDr/jira-magdy';
const WORKFLOW = 'sync.yml';
const BRANCH = 'main';
const TIME_ZONE = 'Africa/Cairo';

function triggerSync() {
  const now = new Date();
  const hour = Number(Utilities.formatDate(now, TIME_ZONE, 'H'));
  const minute = Number(Utilities.formatDate(now, TIME_ZONE, 'm'));
  const night = hour >= 23 || hour < 9;
  // overnight, only the first trigger of every second hour (odd hours: 23, 1, 3, 5, 7)
  if (night && (hour % 2 === 0 || minute >= 30)) return;

  const token = PropertiesService.getScriptProperties().getProperty('GITHUB_TOKEN');
  if (!token) throw new Error('Add GITHUB_TOKEN under Project Settings > Script properties');

  const response = UrlFetchApp.fetch(
    'https://api.github.com/repos/' + REPO + '/actions/workflows/' + WORKFLOW + '/dispatches',
    {
      method: 'post',
      contentType: 'application/json',
      headers: {
        Authorization: 'Bearer ' + token,
        Accept: 'application/vnd.github+json',
        'X-GitHub-Api-Version': '2022-11-28',
      },
      payload: JSON.stringify({ ref: BRANCH }),
      muteHttpExceptions: true,
    });
  if (response.getResponseCode() !== 204) {
    throw new Error('GitHub answered ' + response.getResponseCode() + ': ' + response.
    getContentText());
  }
}
```

D GitHub Workflow

The file `.github/workflows/sync.yml`. The secrets are referenced by name only.

```
name: Sync Jira to Google Sheets

on:
  # Started by the Google Apps Script timer or by hand from the Actions tab.
  workflow_dispatch:

concurrency:
  group: sync-sheets
  cancel-in-progress: false

jobs:
  sync:
    runs-on: ubuntu-latest
    timeout-minutes: 9
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
          cache: pip
      - run: pip install -r requirements.txt
      - run: python bo_sheets.py
        env:
          JIRA_BASE_URL: https://connect-money.atlassian.net
          JIRA_EMAIL: <the Jira login email>
          JIRA_API_TOKEN: ${{ secrets.JIRA_API_TOKEN }}
          GOOGLE_SERVICE_ACCOUNT_JSON: ${{ secrets.GOOGLE_SERVICE_ACCOUNT_JSON }}
          BO_SHEET_ID: <id of BO Tickets (Auto)>
          INPROGRESS_SHEET_ID: <id of In Progress>
```

The `concurrency` setting makes sure two runs never write to the sheets at the same time: a second run waits for the first.